

Cheatsheet 1 - Concevoir et lire les cardinalités

Partir des règles, puis représenter les objets et leurs liens. Lire chaque cardinalité depuis une occurrence de l'entité près de laquelle elle est écrite.

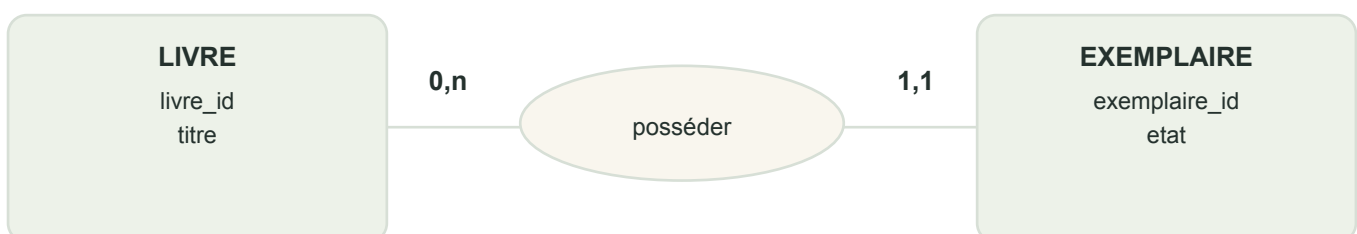
Vocabulaire et point de départ

- Entité : type d'objet, comme LIVRE. Occurrence : livre précis, comme le livre 7.
- Attribut : propriété, comme titre. Identifiant : valeur distinguant les occurrences, comme livre_id.
- Association : lien entre objets. Dictionnaire : sens, domaine et obligations des données.
- Story : En tant que [acteur], je souhaite [action], afin de [bénéfice]. Critère : situation précise → résultat vérifiable. Règle : obligation métier.

Minimum, maximum, signification

Cardinalité	Lecture pour une occurrence
0,1	Facultatif ; au plus un lien.
1,1	Obligatoire ; exactement un lien.
0,n	Facultatif ; plusieurs liens possibles.
1,n	Obligatoire ; au moins un lien.

LIVRE peut avoir zéro copie ; EXEMPLAIRE doit appartenir à un livre



Un livre : de zéro à plusieurs exemplaires.

Un exemplaire : exactement un livre.

Deux pièges à reconnaître

Un acteur n'est pas automatiquement une entité. Une association binaire relie deux types d'entités ; une ternaire relie trois objets dans un même fait. Avec Alice-SQL-C1, Alice-JavaScript-C2 et Bob-SQL-C2, les paires seules peuvent inventer Alice-SQL-C2 : conserver le triplet ou prouver une décomposition sans perte.

RÉFÉRENCES · [09] Mocodo · Manuel de référence de l'auteur

[01] Agile Alliance · Modèle de user story

[03] Sibel Adali, RPI · Dépendances, formes normales et décomposition

Cheatsheet 2 - Traduire en MLD puis en MPD

MCD : objets et règles. MLD : relations et clés ; MPD : types, contraintes et choix du moteur.

Les clés et leurs rôles

PK, clé primaire : identifie une ligne. FK, clé étrangère : référence une clé existante. Dans les exemples ci-dessous, les identifiants sont entiers et une relation de liaison conserve un lien entre les autres relations.

Quatre passages à mémoriser

Situation	Traduction
1:n : un livre possède plusieurs copies ; chaque copie a un livre.	FK livre_id dans EXEMPLAIRE ; NOT NULL si obligatoire.
n:n : commandes et produits ; au plus une ligne par couple.	LIGNE avec deux FK ; PK du couple. Quantité dans LIGNE.
1:1 : personne 0,1 ; dossier 1,1.	FK personne_id dans DOSSIER, NOT NULL et UNIQUE.
Ternaire : enseignant-matière-classe ; seul le triplet est unique.	AFFECTATION avec trois FK ; PK des trois identifiants.

Contraintes du MPD PostgreSQL

Écriture	Garantie
PRIMARY KEY	Unicité et présence de l'identifiant.
FOREIGN KEY / REFERENCES	Existence de la valeur référencée.
NOT NULL	Valeur obligatoire.
UNIQUE	Absence de doublon selon la contrainte.
CHECK (quantite > 0)	Condition vraie ou inconnue ; ajouter NOT NULL pour interdire l'absence.

Vérifications avant de créer les tables

- Nommer les types et les règles de suppression des références.
- Une FK seule n'impose pas au parent d'avoir au moins un enfant.
- Un prêt historique répété demande un identifiant d'événement ou une clé adaptée ; le seul couple lecteur-exemplaire peut être insuffisant.

RÉFÉRENCES · [09] Mocodo · Manuel de référence de l'auteur
[23] PostgreSQL · Contraintes

Cheatsheet 3 - Normalisation et cas d'utilisation

Une dépendance fonctionnelle $X \rightarrow Y$ impose le même Y pour le même X. Les règles doivent rester vraies dans tous les états valides.

Clés et attributs

Superclé : ensemble identifiant une ligne. Candidate : superclé minimale. Primaire : candidate choisie. Attribut premier : appartient à au moins une candidate. Examiner toutes les candidates ; ajouter un id artificiel ne supprime pas les autres dépendances.

Formes normales du modèle relationnel classique

Forme	Question de contrôle
1NF	Une valeur atomique par champ, sans groupe répété ni collection ?
2NF	1NF ; aucun non-premier dépend d'une partie stricte d'une candidate ?
3NF	Pour $X \rightarrow A$, A hors de X : X superclé ou A premier ?
BCNF	Pour $X \rightarrow A$, A hors de X : X toujours superclé ?

UML : objectifs d'acteurs, distincts du MCD

Lien	Flèche	Cas du distributeur
include	Base $\rightarrow$ inclus	Retirer $\rightarrow$ Vérifier le code ; inclus dans le retrait.
extend	Extension $\rightarrow$ base	Reçu $\rightarrow$ Retirer ; sur demande, à un point prévu. Retrait complet sans reçu.

Réflexes de correction

- Dépendance partielle : (commande, produit) $\rightarrow$ quantité, mais produit $\rightarrow$ libellé. Séparer PRODUIT et LIGNE.
- Dépendance transitive : commande $\rightarrow$ client $\rightarrow$ nom. Séparer CLIENT et COMMANDE.
- Après décomposition, vérifier reconstruction sans perte et conservation des dépendances : deux propriétés différentes.
- Pour un cas d'utilisation, préciser acteur, préconditions, scénario, alternatives et résultat ; l'acteur ne devient pas automatiquement une table.

RÉFÉRENCES · [19] Stéphane Crozat · Théorie de la normalisation relationnelle
 [03] Sibel Adali, RPI · Dépendances, formes normales et décomposition
 [22] OMG · UML 2.5.1, standard et document normatif, chapitre 18

TypeScript · Mémo de lecture

Lire les types comme des possibilités, puis vérifier ce qui vient de l'extérieur.

Repères essentiels

À reconnaître	À expliquer
interface / type	Forme d'objet ; alias possible pour objet, union, tuple.
T[] / [A, B] / readonly	Liste ; positions fixes ; modification interdite via ce type.
unknown / any / as	Vérifier ; retirer les contrôles ; affirmer sans valider.
status: 'ok' 'error'	Discriminant : sélectionner la bonne variante.
<T> / Partial / Pick / Omit	Conserver une relation de types ; construire un type.
async / await / import type	Promise ; attendre son résultat ; import effacé.

Commandes dans un dossier d'essai

```
npm install -D typescript tsx
npx tsc --strict --noEmit exemple.ts
npx tsx exemple.ts
# Compilation et exécution de deux modules :
npx tsc --strict --target ES2022 --module NodeNext --outDir dist main.ts
node dist/main.js
```

Avant de dire « c'est sûr »

- Les types ne valident pas les données reçues et ne remplacent pas les tests.
- Prévoir null, undefined, listes vides et erreurs ; vérifier le domaine métier.
- strict peut être conservé dans tsconfig.json avec compilerOptions.strict: true.
- Une assertion as contourne un contrôle : préférer une preuve réelle avec une garde.

RÉFÉRENCES · [\[37\]](#) [\[39\]](#) [\[38\]](#) [\[40\]](#) [\[41\]](#) [\[20\]](#) · Notices complètes à la fin ; liens cliquables.

SQL · Mémo de lecture et de contrôle

Demander quelles lignes chaque étape conserve, puis expliquer l'intégrité et les effets d'une écriture.

Repères

Écriture	Question
PK / FK / NOT NULL / UNIQUE / CHECK	Identité ? Référence ? Présence ? Doublet ? Règle ?
SELECT / WHERE / ORDER BY / LIMIT / OFFSET	Quelles colonnes, lignes, ordre et page ?
INNER / LEFT JOIN ; IS NULL	Correspondances seules ou lignes absentes conservées ?
COUNT(col) / GROUP BY / HAVING	Quelles valeurs non NULL et quels groupes ?
EXISTS / paramètres / RETURNING	Une ligne existe ? Valeur séparée ? Résultat de ma mutation ?
Transaction / index / EXPLAIN	Atomicité ? Coût des accès ? Plan prévu ou mesuré ?

Fragment à lire avec une commande id 10, total 30

```
BEGIN;
UPDATE commande SET total = 40 WHERE id = 10;
ROLLBACK; -- Le total revient à 30.
-- Lire avec des paramètres :
-- SELECT id, total FROM commande WHERE id = $1
-- Fournir [10] au pilote, sans concaténation.
```

Questions de vérification

- LEFT JOIN : une ligne sans enfant n'est pas un enfant à compter.
- CHECK ne rend pas automatiquement un champ obligatoire ; ajouter NOT NULL.
- WHERE filtre les lignes ; HAVING filtre les groupes. COALESCE ne confond pas 0 et NULL.
- EXPLAIN ANALYZE exécute réellement ; une mutation mesurée peut changer les données.
- Un index accélère certains accès mais consomme de l'espace et ralentit certaines écritures.

RÉFÉRENCES · [\[23\]](#) [\[31\]](#) [\[30\]](#) [\[29\]](#) [\[24\]](#) [\[25\]](#) [\[26\]](#) [\[32\]](#) [\[28\]](#) [\[27\]](#) [\[33\]](#) · Notices complètes à la fin ; liens cliquables.

Mémo Docker

Image = paquet ; conteneur = instance ; volume = stockage indépendant. Avant une commande, repérer sa cible.

Commandes et résultats

Commande	Effet
<code>docker build -t atelier-api .</code>	Fabriquer une image depuis le dossier.
<code>docker run --name api atelier-api</code>	Créer et démarrer un conteneur.
<code>docker ps -a</code> / <code>docker logs api</code>	Voir les états / lire les messages.
<code>docker stop api</code> / <code>docker start api</code>	Arrêter / redémarrer le même conteneur.
<code>docker compose up -d</code> / <code>ps</code>	Lancer les services / examiner leurs états.
<code>docker compose down</code>	Retirer conteneurs et réseau ; garder les volumes nommés.

Un port se lit hôte → conteneur

Contexte	Adresse
Publication API 8080:3000	Machine : localhost:8080
API vers le service db	db:5432
Publication DB 15432:5432	Machine : localhost:15432

Ordre usuel avec des dépendances npm

```
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
CMD ["node", "api.mjs"]
```

Pièges fréquents

- Fragment ci-dessus : suppose des fichiers npm cohérents et `api.mjs` ; il illustre le cache.
- `RUN` agit au build ; `CMD` au démarrage. `EXPOSE` ne publie aucun port.
- `localhost` dans un conteneur vise ce conteneur. Une base démarrée peut encore être indisponible.
- Volume nommé : persistance, pas historique de sauvegarde. `down -v` supprime ses données.
- Erreur réseau : vérifier état, logs, nom du service, port interne et configuration.

RÉFÉRENCES · [\[07\]](#) [\[04\]](#) [\[05\]](#) [\[06\]](#) · Notices complètes à la fin ; liens cliquables.

Mémo tests JavaScript

Préparer un contexte connu, exercer le comportement, comparer à un attendu issu de la règle.

Unitaire

Une règle
Dépendances remplacées

Intégration

Composants réunis
Base ou HTTP réels

Parcours

Action utilisateur
Résultat visible

Assertions à choisir

Intention	Node	Vitest
Valeur / identité	<code>assert.strictEqual</code>	<code>expect(...).toBe</code>
Objet ou tableau	<code>assert.deepStrictEqual</code>	<code>expect(...).toEqual</code>
Exception immédiate	<code>assert.throws</code>	<code>expect(fn).toThrow</code>
Rejet asynchrone	<code>await assert.rejects</code>	<code>await expect(p).rejects.toThrow</code>

Vitest préinstallé : `titres.test.mjs`, puis `npx vitest run`

```
import { test, expect } from "vitest";
const pending = tasks => tasks.filter(t => !t.done);
test("garde les tâches non terminées", () => {
  const tasks = [{id:1, done:false}, {id:2, done:true}];
  const result = pending(tasks);
  expect(result).toEqual([{id:1, done:false}]);
});
```

Routine de vérification

- AAA : Arrange = contexte ; Act = appel ; Assert = attendu.
- Cas : succès, absence, bornes, valeurs invalides, erreurs et permissions.
- HTTP : vérifier statut et corps. `await` attend la requête et ses assertions.
- Fixtures isolées ; doubles pour la logique ; vraie base dédiée pour l'intégration. Nettoyer avec `after/afterEach` ou `t.after`.
- TDD : Red → Green → Refactor. Une correction de bug mérite un test de non-régression.
- Couverture élevée : code parcouru ; elle ne garantit pas la qualité des assertions.

RÉFÉRENCES · [12] [11] [21] [10] · Notices complètes à la fin ; liens cliquables.

Mémo backend et API

Transport → règles → données. Chaque requête vérifie ses entrées et ses permissions ; une interface masquée ne protège pas le serveur.



La réponse revient vers le navigateur : statut HTTP et données utiles.

Codes HTTP : le contrat précise leur usage

Code	Sens
200 / 201	Succès / création réalisée
400 / 422	Requête incorrecte / contenu compris mais invalide
401	Identité à établir ; défi WWW-Authenticate
403 / 404	Interdit / ressource introuvable
409	Conflit avec l'état actuel
500	Incident interne ; réponse sans secrets

Accès aux données : Prisma 6.19.0

Notion	Repère
SQL paramétré	SELECT ... WHERE id = \$1 ; valeur passée séparément
findMany / findUnique	Tableau, même vide / ligne ou null
where / select / include	Filtrer / choisir les champs / ajouter une relation
orderBy / skip / take	Trier / décaler / limiter
migrate dev / migrate deploy	Écrire et appliquer / appliquer les migrations existantes
generate / transaction	Générer le client / tout valider ou tout annuler

Points de contrôle

- Middleware : répondre ou next(). express.json avant les routes ; erreur à quatre paramètres en fin de chaîne. Express 5 transmet le rejet d'un handler async retourné.
- Contrôleur : HTTP ; service : règles ; repository : persistance.
- TypeScript ne valide pas un JSON reçu. Zod safeParse vérifie à l'exécution ; utiliser data après succès.
- JWT : verify, pas decode ; contenu du jeton lisible, sans chiffrement implicite. Vérifier les droits sur chaque ressource.
- Mot de passe : hash adapté et sel. Cookie : HttpOnly, Secure, protection CSRF.
- Prisma et @prisma/client : version 6.19.0 pour ces exemples. Contraintes en base ; fixtures isolées pour les tests.

RÉFÉRENCES · [08] [02] [42] [36] [35] [34] · Notices complètes à la fin ; liens cliquables.

React · Mémo de données et d'effets

D'abord expliquer l'état attendu de l'écran ; ensuite distinguer son calcul du travail extérieur.

Six repères

Notion	À vérifier
Composant / props	Fonction d'interface / entrées fournies par le parent.
State / updater	Valeur conservée / prochain état depuis le précédent.
Formulaire contrôlé	value + onChange ; une seule source de la valeur.
Liste / état partagé	key stable ; propriétaire dans l'ancêtre commun.
useEffect	Dépendances réactives, nettoyage et erreurs.
Router	Chemin → composant ; useParams lit les segments variables.

Fragments à lire ; imports et id viennent du composant

```
const [name, setName] = useState('');
<input value={name} onChange={e => setName(e.target.value)} />
// L'effet lit id ; document est fourni par le navigateur.
useEffect(() => {
  const previousTitle = document.title;
  document.title = `Activité ${id}`;
  return () => { document.title = previousTitle; };
}, [id]);
```

Checklist de compréhension

- Rendu : décrire l'écran sans lancer un appel réseau à chaque calcul.
- Async : attente, succès, vide, erreur ; await dans une fonction async interne.
- Nettoyer abonnement ou opération ; éviter les résultats périmés.
- Modifier objets et tableaux sans mutation directe ; conserver des identifiants stables.
- Navigation : contrôle d'accès supplémentaire sur le serveur pour les données privées.

RÉFÉRENCES · [13] [17] [15] [14] [16] [18] · Notices complètes à la fin ; liens cliquables.

Lire les références

Les situations sont créées pour apprendre. Les liens ci-dessous donnent les références des notions : documentation des outils, spécification UML ou travaux académiques selon le sujet.

[01] Agile Alliance · Modèle de user story

Documentation officielle

<https://agilealliance.org/glossary/user-story-template/>

[02] OWASP · vérifier les permissions

Documentation officielle

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

[03] Sibel Adali, RPI · Dépendances, formes normales et décomposition

Référence académique

https://cs.rpi.edu/~sibel/csci4380/fall2020/course_notes/normalization.html

[04] Docker Compose · réseau et noms de services

Documentation officielle

<https://docs.docker.com/compose/how-tos/networking/>

[05] Docker · volumes persistants

Documentation officielle

<https://docs.docker.com/engine/storage/volumes/>

[06] Docker Compose · arrêt et suppression

Documentation officielle

<https://docs.docker.com/reference/cli/docker/compose/down/>

[07] Docker · instructions du Dockerfile

Documentation officielle

<https://docs.docker.com/reference/dockerfile/>

[08] Express 5 · traitement des erreurs

Documentation officielle

<https://expressjs.com/en/guide/error-handling/>

[09] Mocodo · Manuel de référence de l'auteur

Documentation officielle

https://laowantong.github.io/mocodo/doc/fr_refman.html

[10] Microsoft Learn · cycle Red, Green, Refactor

Documentation officielle

<https://learn.microsoft.com/en-us/aspnet/mvc/overview/older-versions-1/contact-manager/iteration-6-use-test-driven-development-cs>

Références · suite

[11] Node.js · vérifier un résultat avec assert

Documentation officielle

<https://nodejs.org/api/assert.html>

[12] Node.js · exécuter des tests

Documentation officielle

<https://nodejs.org/api/test.html>

[13] React · Transmettre des props

Documentation officielle

<https://react.dev/learn/passing-props-to-a-component>

[14] React · Partager un état

Documentation officielle

<https://react.dev/learn/sharing-state-between-components>

[15] React · Champs input

Documentation officielle

<https://react.dev/reference/react-dom/components/input>

[16] React · useEffect et nettoyage

Documentation officielle

<https://react.dev/reference/react/useEffect>

[17] React · useState

Documentation officielle

<https://react.dev/reference/react/useState>

[18] React Router · useParams

Documentation officielle

<https://reactrouter.com/api/hooks/useParams>

[19] Stéphane Crozat · Théorie de la normalisation relationnelle

Référence académique

<https://stph.scenari-community.org/bdd/nor1.pdf>

[20] tsx · Exécution et vérification des types

Documentation officielle

<https://tsx.is/typescript>

[21] Vitest · comparer les résultats

Documentation officielle

<https://vitest.dev/api/expect.html>

Références · suite

[22] OMG · UML 2.5.1, standard et document normatif, chapitre 18

Norme / spécification

<https://www.omg.org/spec/UML/2.5.1>

[23] PostgreSQL · Contraintes

Documentation officielle

<https://www.postgresql.org/docs/current/ddl-constraints.html>

[24] PostgreSQL · Comparaisons et NULL

Documentation officielle

<https://www.postgresql.org/docs/current/functions-comparison.html>

[25] PostgreSQL · COALESCE et expressions conditionnelles

Documentation officielle

<https://www.postgresql.org/docs/current/functions-conditional.html>

[26] PostgreSQL · Sous-requêtes

Documentation officielle

<https://www.postgresql.org/docs/current/functions-subquery.html>

[27] PostgreSQL · Index

Documentation officielle

<https://www.postgresql.org/docs/current/indexes.html>

[28] PostgreSQL · PREPARE et paramètres

Documentation officielle

<https://www.postgresql.org/docs/current/sql-prepare.html>

[29] PostgreSQL · Agrégats

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-agg.html>

[30] PostgreSQL · Jointures

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-join.html>

[31] PostgreSQL · Interroger une table

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-select.html>

[32] PostgreSQL · Transactions

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-transactions.html>

Références · suite

[33] PostgreSQL · EXPLAIN

Documentation officielle

<https://www.postgresql.org/docs/current/using-explain.html>

[34] Prisma ORM 6 · transactions

Documentation officielle

<https://www.prisma.io/docs/orm/v6/prisma-client/queries/transactions>

[35] Prisma ORM 6 · migrations en développement et déploiement

Documentation officielle

<https://www.prisma.io/docs/orm/v6/prisma-migrate/workflows/development-and-production>

[36] Prisma ORM 6 · méthodes du client

Documentation officielle

<https://www.prisma.io/docs/orm/v6/reference/prisma-client-reference>

[37] TypeScript · Types courants

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>

[38] TypeScript · Génériques

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/generics.html>

[39] TypeScript · Narrowing et unions discriminées

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html>

[40] TypeScript · Types utilitaires

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/utility-types.html>

[41] TypeScript · Référence tsconfig

Documentation officielle

<https://www.typescriptlang.org/tsconfig/>

[42] Zod · validation des données à l'exécution

Documentation officielle

<https://zod.dev/basics>